

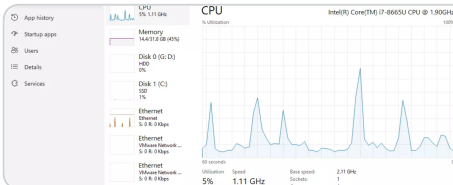
向量数据库Milvus使用说明

相关代码链接



https://github.com/jwj1111/milvus_guidance_vector_database

1. 通过Docker容器运行，官方提供了dockerstandalone.bat的安装脚本。管理员启动docker desktop，在指定cd运行以下命令，下载“安装脚本”



https://milvus.io/docs/zh/install_standalone-windows.md

Run Milvus in Docker (Linux) | Milvus Documentation

Learn how to install Milvus standalone with Docker Desktop for Windows. | v2.6.x

代码块

```
1 cd D:\milvus_standalone
2 Invoke-WebRequest https://raw.githubusercontent.com/milvus-io/milvus/refs/heads/master/scripts/standalone_embed.bat -OutFile standalone.bat
```

2. 通过脚本启动/关闭/删除milvus standalone

3. 运行下载脚本，将 Milvus 作为 Docker 容器启动。

```
C:\>standalone.bat start
Wait for Milvus starting...
Start successfully.
To change the default Milvus configuration, edit user.yaml and restart the service.
```

运行安装脚本后

- 名为Milvus-standalone的 docker 容器已在19530 端口启动。
- 嵌入式 etcd 与 Milvus 安装在同一个容器中，服务端口为2379。其配置文件被映射到当前文件夹中的embedEtcd.yaml。
- Milvus 数据卷映射到当前文件夹中的volumes/milvus。

可以使用以下命令管理 Milvus 容器和存储的数据。

```
# Stop Milvus
C:\>standalone.bat stop
Stop successfully.

# Delete Milvus container
C:\>standalone.bat delete
Delete Milvus container successfully. # Container has been removed.
Delete successfully. # Data has been removed.
```

代码块

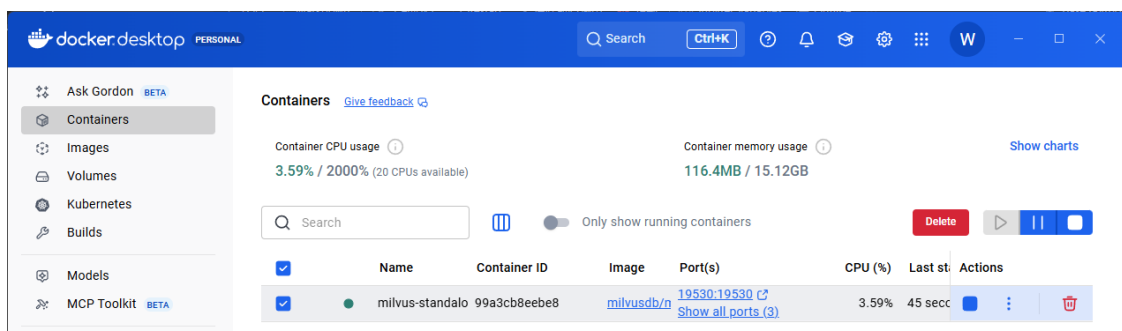
```
1 cd D:\milvus_standalone
2 standalone.bat start
```

启动后查看当前docker状态

部署服务器请先确保以下公网端口已经放开

```
(base) PS C:\Users\LK> cd D:\milvus_standalone
(base) PS D:\milvus_standalone> standalone.bat start
Unable to find image 'milvusdb/milvus:v2.6.8' locally
v2.6.8: Pulling from milvusdb/milvus
de6368d16466: Pull complete
Wait for Milvus Starting...
Start successfully.
To change the default Milvus configuration, edit user.yaml and restart the service.
(base) PS D:\milvus_standalone> docker ps -a
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
99a3cb8eebe8   milvusdb/milvus:v2.6.8              "/tiny -- milvus run..." 2 minutes ago  Up 2 minutes (healthy)   0.0.0.0:2379->2379/tcp, [::]:2379->2379/tcp, 0.0.0.0:9091->9091/tcp, [::]:9091->9091/tcp, 0.0.0.0:19530->19530/tcp, [::]:19530->19530/tcp
(base) PS D:\milvus_standalone>
```

- 初次配置后可以通过docker desktop启动（请确保后台启动了WSL Service使docker desktop正常运行）



代码块

```
1 cd D:\milvus_standalone
2 standalone.bat stop
```

代码块

```
1 cd D:\milvus_standalone
2 standalone.bat delete
```

3. milvus可视化工具



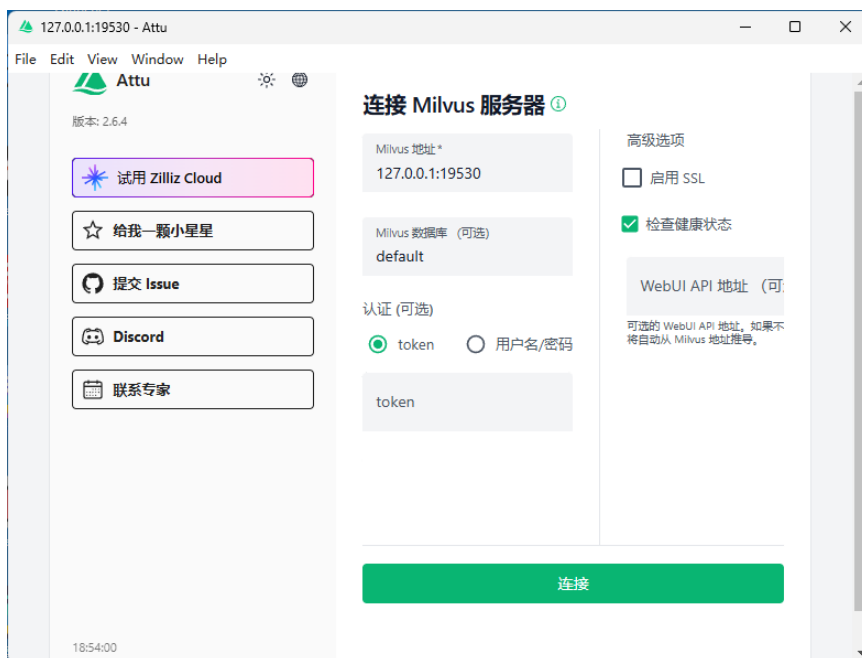
<https://zilliz.com.cn/attu>

Attu - Milvus 最佳 GUI 工具

Attu 是 Milvus 向量数据库的首选 GUI 工具。通过直观的界面简化向量数据库管理，包括集合管理、向量搜索和系统监控。

4. 默认情况下IP是放开无鉴权的。所以需要手动开启鉴权

无鉴权belike获得端口便可直接连接：



增加鉴权！！

- 方式1：在docker容器内修改

代码块

```
1 # 使用容器ID进入容器内部
```

```

2  docker exec -it 99a3cb8eebe8 /bin/bash
3  # 内部配置目录
4  cd /milvus/configs
5  # 此时可以看见milvus.yaml
6  ls
7  # 修改milvus.yaml
8  '''
9  common:
10     security:
11         authorizationEnabled: true
12  '''

```

- 方式2: user.yaml的参数会直接作用于docker容器内部的milvus.yaml

```

! user.yaml X
D: > milvus_standalone > ! user.yaml
1  # Extra config to override default milvus.yaml
2  common:
3      security:
4          authorizationEnabled: true

```

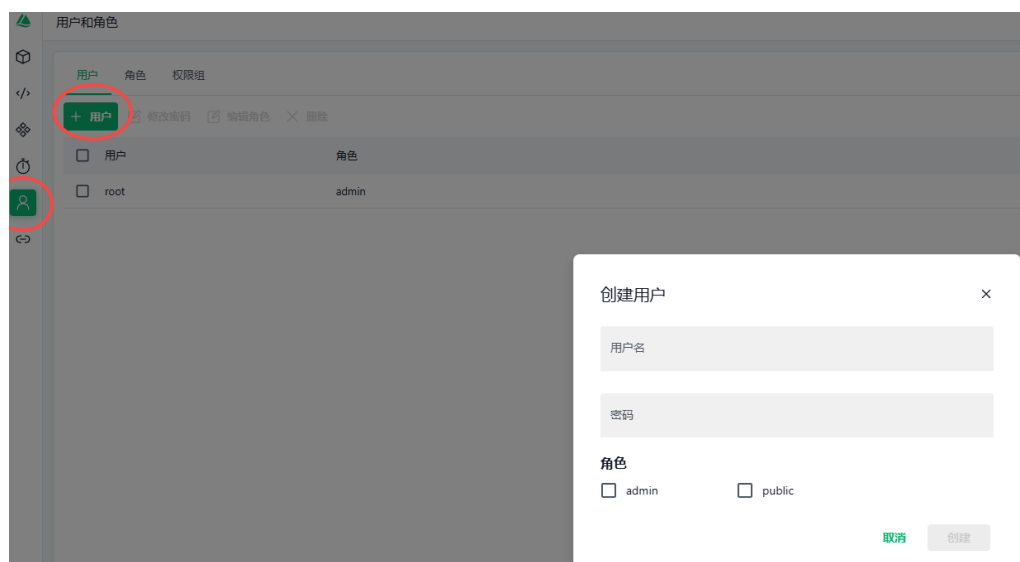
默认用户名为: root

密码为: Milvus

token为: root:Milvus

鉴权用户管理

attu可视化设置



pymilvus密码更新

- 通过客户端 (似乎存在延迟)

```
from pymilvus import MilvusClient
✓ 0.0s

# 初始化客户端
client = MilvusClient(host="127.0.0.1", port="19530", token="root:Milvus")
✓ 0.0s

client.update_password(user_name="root", old_password="Milvus", new_password="www")
```

- 连接并修改（修改密码/添加新用户）

```
pymilvus准备了方便的数据接口与用户管理操作

from pymilvus import connections, utility

# 连接数据库
connections.connect(uri="http://localhost:19530", token="root:Milvus")
# connections.connect(host="127.0.0.1", port="19530", token="root:Milvus")

# 更新密码
utility.update_password(user_name="root", old_password="Milvus", new_password="www")

# 创建新用户
utility.create_user(user="aaa", password="www")
```

5. pymilvus内部直接封装了快速调用embedding/reranker模型的方法

代码块

```
1 # 首先需要单独安装pymilvus的模型模块
2 pip install "pymilvus[model]"
```

需要单独注意其中的onnxruntime，在windows需要匹配环境。如果出现dll报错

代码块

```
1 # 先单独卸载onnxruntime
2 pip uninstall onnxruntime onnxruntime-gpu
3 # conda相关环境
4 conda install -c conda-forge vs2015_runtime ucrt vc
5 # 重新安装onnxruntime (cpu)
6 # pip install onnxruntime
7 # GPU (与cpu二选一)
```

```
8 pip install onnxruntime-gpu
```

- embedding以常规sentence-transformer为例（粗召回语义匹配）

代码块

```
1 from pymilvus import model
2 # 初始化嵌入模型实例
3 sentence_trans_ef = model.dense.SentenceTransformerEmbeddingFunction(
4     model_name="aaa", # sen_trans模型
5     device="cuda:0" #'cpu' or 'cuda:0'
6 )
7 # 可直接准备原始文本
8 docs = [
9     "Artificial intelligence was founded as an academic discipline in 1956.",
10    "Alan Turing was the first person to conduct substantial research in AI.",
11    "Born in Maida Vale, London, Turing was raised in southern England.",
12 ]
13 # 可直接嵌入为List[np.array]
14 docs_embeddings = sentence_trans_ef.encode_documents(docs)
15 # 维度与向量shape
16 print("Dim:", sentence_trans_ef.dim, docs_embeddings[0].shape)
```

- reranker以常规cross-encoder reranker为例（精召回语义匹配）

代码块

```
1 from pymilvus.model.reranker import CrossEncoderRerankFunction
2 # 初始化reranker实例
3 ce_rf = CrossEncoderRerankFunction(
4     model_name="aaa", # cross_encoder_reranker
5     device="cuda:0" #'cpu' or 'cuda:0'
6 )
7 # 提取prompt与知识库内容
8 query = "What event in 1956 marked the official birth of artificial
9 intelligence as a discipline?"
10
11 documents = [
12     "In 1950, Alan Turing published his seminal paper, 'Computing Machinery
13 and Intelligence,' proposing the Turing Test as a criterion of intelligence, a
14 foundational concept in the philosophy and development of artificial
15 intelligence.",
16     "The Dartmouth Conference in 1956 is considered the birthplace of
17 artificial intelligence as a field; here, John McCarthy and others coined the
18 term 'artificial intelligence' and laid out its basic goals.",
19 ]
```

```

13     "In 1951, British mathematician and computer scientist Alan Turing also
    developed the first program designed to play chess, demonstrating an early
    example of AI in game strategy.",
14     "The invention of the Logic Theorist by Allen Newell, Herbert A. Simon,
    and Cliff Shaw in 1955 marked the creation of the first true AI program, which
    was capable of solving logic problems, akin to proving mathematical theorems."
15 ]
16 # 可直接输出k个List[Tuple(id, score, document)]
17 reranker_results = ce_rf(query=query, documents=documents, top_k=3)
18 # belike
19 for result in results:
20     print(f"Index: {result.index}")
21     print(f"Score: {result.score:.6f}")
22     print(f"Text: {result.text}\n")

```

- embedding内置了BGE、GTE、Jina等
- reranker内置了BGE、Jina等
- 这些便捷模型不仅可以直接使用，还可以作为schema或hybrid_search的参数直接传入milvus
 - 在内置自动嵌入时，比DOCKER的HuggingFace TEI更加方便
 - 也可以手动调用后，将向量传入milvus，数据管理更加灵活

6. milvus数据管理

6.1 创建collections

collection本质是一个二维表的结构化集合

为实现每列一个字段，每行一个实体的管理，需要定义schema。要插入的每个实体都必须符合schema的定义。

除了schema，还可以定义索引参数提高向量搜索的效率。

创建一个collection，需要：

- 创建schema
- 设置索引（可选，向量field必须）
- 创建collection

6.1.1 创建schema

schema用来定义collection的数据结构，为每一个field（列）设置名称与数据结构要求。

- 每个collection必须有一个唯一不重复的主键（常为id，INT64），推荐使用autoid，除非手动设置是有益的

- `enable_dynamic_field` 动态字段，允许在插入数据时，插入未在schema中定义的字段。默认建议为False

代码块

```
1 from pymilvus import MilvusClient, DataType
2 client = MilvusClient(uri="http://localhost:19530", token="root:Milvus")
```

代码块

```
1 # 定义schema
2 schema = MilvusClient.create_schema() # 推荐默认设置 (auto_id=False,
    enable_dynamic_field=False) , 可在具体field中精确定义
3 # schema = MilvusClient.create_schema(
4 #     auto_id=True,
5 #     enable_dynamic_field=False,
6 # )
```

代码块

```
1 # 添加主字段 (id)
2 schema.add_field(field_name="my_id", datatype=DataType.INT64, is_primary=True,
    auto_id=True)
3 # schema.add_field(field_name="product_id", datatype=DataType.VARCHAR,
    is_primary=True, auto_id=False, max_length=50)
```

向量类型包括

- dense vector: `FLOAT_VECTOR`、`FLOAT16_VECTOR`、`BFLOAT16_VECTOR`、`INT8_VECTOR`
- sparse vector: `SPARSE_FLOAT_VECTOR`
- binary vector: `BINARY_VECTOR`(更适合图像处理)

代码块

```
1 # 添加向量
2 schema.add_field(field_name="dense", datatype=DataType.FLOAT_VECTOR, dim=5) #
    must dim
3 schema.add_field(field_name="sparse", datatype=DataType.SPARSE_FLOAT_VECTOR) #
    no dim
4 # 添加标量
5 schema.add_field(field_name="text", datatype=DataType.VARCHAR, max_length=200)
    # str
6 schema.add_field(field_name="myint", datatype=DataType.INT64) # int
7 schema.add_field(field_name="yes_no", datatype=DataType.BOOL) # bool
```



```
8 # 添加复合字段
9 schema.add_field(field_name="json", datatype=DataType.JSON)
10 schema.add_field(field_name="array", datatype=DataType.ARRAY,
    element_type=DataType.VARCHAR, max_capacity=5, max_length=512)
```

代码块

```
1 # 删除字段 (在 create_collection 之前)
2 # schema.fields 是一个列表, 直接用列表推导式过滤掉你不想要的字段
3 schema.fields = [f for f in schema.fields if f.name != "temp_field"]
```

6.1.2 创建索引

控制field的检索方式与匹配metric

向量与标量的索引参数不同:

- 向量必须设置索引:
 - dense:
 - metric type: L2 (欧氏距离, 向量直线距离), IP (内积), COSINE (余弦相似度, 适合文本)
 - index type: FLAT, IVF_FLAT, IVF_PQ, IVF_SQ8, HNSW(图), HNSW_SQ, HNSW_PQ, HNSW_PRQ, SCANN, 用于基于 CPU 的 ANN 搜索
 - 其中:
 - FLAT: 简单的线性扫描, 百万级别, 100%召回。
 - IVF_FLAT: IVF 索引 + FLAT 索引。划分nlist个簇, 通过比较聚类中心, 筛出nprobe个簇, 再在nprobe个簇中全部匹配。
 - binary:
 - metric type: JACCARD, HAMMING
 - index type: BIN_FLAT, BIN_IVF_FLAT
 - sparse:
 - metric type: IP, BM25
 - BM25需要搭配参数: `params = {"bm25_k1": 1.2, "bm25_b": 0.75}`。k1词频饱和度, 越小tf上升的边际效应越明显[1.2,2.0]; b文档长度标准化, 默认0.75左右, 1不归一, 0完全归一。
 - index type: SPARSE_INVERTED_INDEX
 - SPARSE_INVERTED_INDEX需要搭配参数: `params = {"inverted_index_algo": "DAAT_MAXSCORE"}`。DAAT_MAXSCORE (默认)、DAAT_WAND、TAAT_NAIVE选一

- 标量可选：INVERTED（倒排索引，神）、BITMAP（位图索引）、STL_SORT（INT8、INT16、INT32、INT64、FLOAT、DOUBLE）、Trie（前缀树索引，快速前缀搜索，VARCHAR）

index type可以直接选择为**AUTOINDEX**自动判断

代码块

```
1  # 定义索引
2  index_params = client.prepare_index_params()
3
4  # 标量索引（如主键）
5  index_params.add_index(
6      field_name="my_id",
7      index_type="AUTOINDEX"
8  )
9
10 # 向量索引（如文本dense）
11 index_params.add_index(
12     field_name="dense",
13     index_name="dense_index",
14     index_type="AUTOINDEX",
15     metric_type="COSINE"
16 )
17 # index_params.add_index(
18 #     field_name="dense",
19 #     index_name="dense_index",
20 #     index_type="IVF_FLAT",
21 #     metric_type="COSINE",
22 #     params={"nlist": 100, "nprobe": 10}
23 # )
24 # 文本sparse
25 # 注意：BM25必须配合方程使用（参考粗召回文档的设置）
26 index_params.add_index(
27     field_name="sparse",
28     index_name="sparse_index",
29     index_type="SPARSE_INVERTED_INDEX",
30     metric_type="BM25",
31     params={"inverted_index_algo": "DAAT_MAXSCORE", "bm25_k1": 1.2, "bm25_b":
32         0.75}
33 )
```

代码块

```
1  # 可在client中删除索引
2  # client.drop_index(collection_name="collection1", index_name="dense_index")
```

6.1.3 创建collections

collections创建包含一系列参数：

- collection_name: 必填
- schema: 必填
- index_params: 有向量时必填
- num_shards: 可选，切片数量。一般建议2亿数据/100gb便增加一个切片
- enable_mmap: 可选，mmap可以结合内存和磁盘，减少内存占用。内容有限时，建议将需要频繁读取的数据全部载入内存，其它可以启用mmap。mmap有全局、collection、field/索引三种级别，默认都为False。优先级field/索引 > collection > 全局。此处为collection的mmap
- consistency_level: 可选，一致性级别。默认为Bounded。可以理解为可检索数据的时效性。新/返回慢→旧/返回快：Strong > Session > Bounded > Eventual。建议使用Strong保证新数据齐全。一致性有collection、search/query两种级别。优先级为search/query > collection。
- properties.collection.ttl.seconds: 可选，集合的过期时间，单位为秒。默认不开启。开启后每过xx秒会把数据清空

前序设置

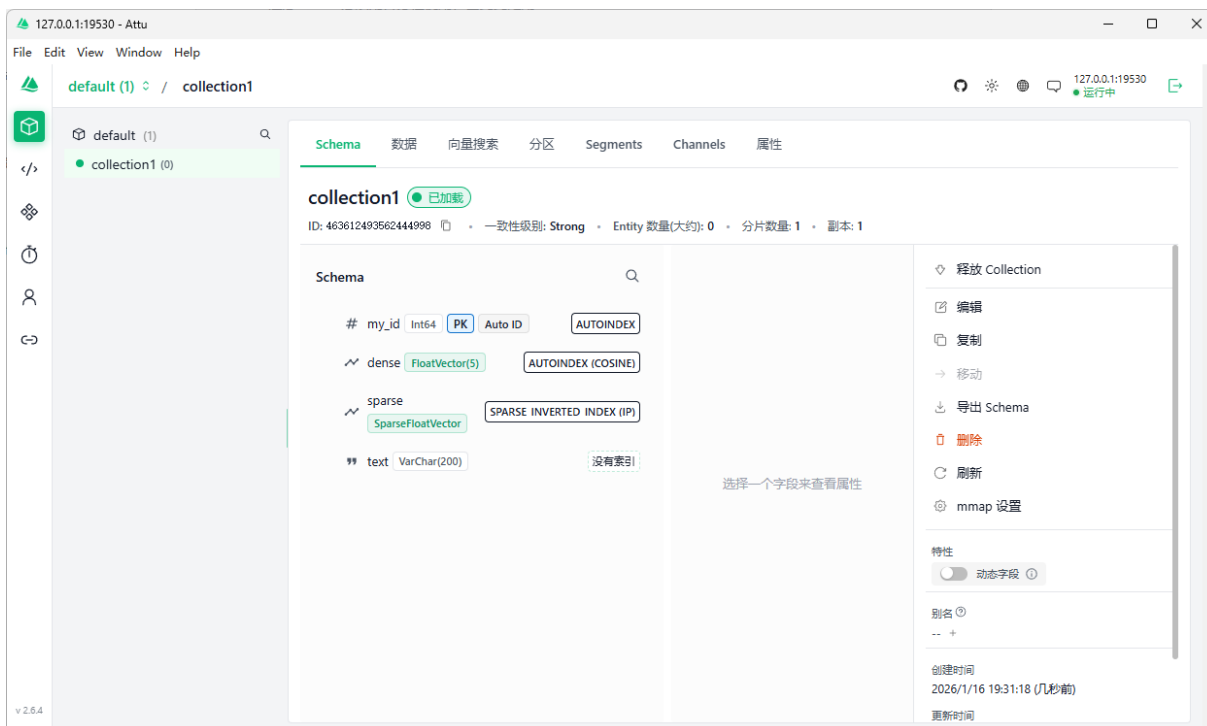
```
1  from pymilvus import MilvusClient, DataType
2
3  client = MilvusClient(uri="http://localhost:19530", token="root:Milvus")
4
5  # 定义schema
6  schema = MilvusClient.create_schema()
7  schema.add_field(field_name="my_id", datatype=DataType.INT64, is_primary=True,
8  auto_id=True)
9  schema.add_field(field_name="dense", datatype=DataType.FLOAT_VECTOR, dim=5)
10 schema.add_field(field_name="sparse", datatype=DataType.SPARSE_FLOAT_VECTOR)
11 schema.add_field(field_name="text", datatype=DataType.VARCHAR, max_length=200)
12
13 # 定义索引
14 index_params = client.prepare_index_params()
15 index_params.add_index(field_name="my_id", index_type="AUTOINDEX")
16 index_params.add_index(field_name="dense", index_name="dense_index",
17 index_type="AUTOINDEX", metric_type="COSINE")
18 # 注意：BM25必须配合方程使用，因此此处用IP
19 index_params.add_index(field_name="sparse", index_name="sparse_index",
20 index_type="SPARSE_INVERTED_INDEX", metric_type="IP", params=
21 {"inverted_index_algo": "DAAT_MAXSCORE"})
```

代码块

```

1  # 创建collection
2  client.create_collection(
3      collection_name="collection1",
4      schema=schema,
5      index_params=index_params,
6      consistency_level="Strong",
7      # shards_num=1,
8      # enable_mmap=False,
9      # properties={
10         #     "collection.ttl.seconds": 999999
11         # }
12  )
13  # 一般只会设置上面四项

```



6.2 加载或释放collections

将collection加载到内存中，或从内存中释放

代码块

```

1  # 加载collection
2  client.load_collection(collection_name="collection1")
3  # 释放collection
4  client.release_collection(collection_name="collection1")
5  # 查看状态
6  res = client.get_load_state(collection_name="collection1")
7  print(res)

```

6.3 修改collections

修改名称、部分属性 (properties={"collection.ttl.seconds / mmap.enabled / timezone": xxxxxx})

代码块

```
1  # collection重命名
2  client.rename_collection(old_name="collection1", new_name="collection11")
3
4  # 修改属性
5  client.alter_collection_properties(
6      collection_name="collection1",
7      properties={"timezone": "Asia/Shanghai"}
8  )
```

6.3.1 添加field字段

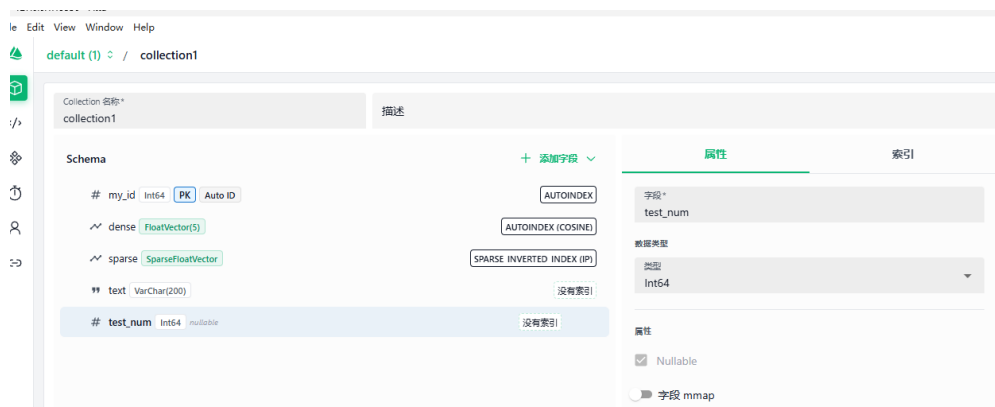
可以给collection添加一个新字段：

- collection_name：必填
- field_name：必填
- （和schema定义时一样的参数）
- nullable：True。新增field时必须设置为True
- default_value：新增field时，如果设置那么field所有数据为该值

代码块

```
1  client.add_collection_field(
2      collection_name="collection1",
3      field_name="new_int",
4      data_type=DataType.INT64,
5      nullable=True,
6      default_value=50
7  )
```

在attu上也可以创建新的field



test_num由添加字段创建，但无法删除

6.4 删除collections/properties

删除collection或重置某项属性

代码块

```
1  # 删除collection
2  client.drop_collection(collection_name="collection1")
3
4  # 删除重置属性
5  client.drop_collection_properties(
6      collection_name="collection1",
7      property_keys=[
8          "collection.ttl.seconds"
9      ]
10 )
```

6.5 写入数据

请确保每条数据与collection的schema一致（主键启用autoID后可以不用传入id）

- 主键启动了autoID时：insert、upsert的数据不传入id（传了也没用），均会自动上传数据并生成主键
- 主键未启动autoID时：insert、upsert的数据必须包含id。insert遇到相同id会报错，upsert遇到相同id会覆盖原数据。

代码块

```
1  # 插入数据insert, autoid开启
2
3  # 数据
4  data = [
5      {"dense": [0.3580376395471989, -0.6023495712049978, 0.18414012509913835,
6          -0.26286205330961354, 0.9029438446296592], "sparse": {1: 0.5, 100: 0.3, 500:
7          0.8}, "text": "这是一个例子。"},
```

```

6     {"dense": [0.19886812562848388, 0.06023560599112088, 0.6976963061752597,
0.2614474506242501, 0.838729485096104], "sparse": {10: 0.1, 200: 0.7, 1000:
0.9}, "text": "这又是一个例子。"}
7 ]
8 # 插入数据
9 res = client.insert(
10     collection_name="collection1",
11     data=data
12 )
13 print(res)
14 # 还可以选择具体的分区
15 # res = client.insert(
16 #     collection_name="collection1",
17 #     data=data,
18 #     partition_name="partition1"
19 # )

```



代码块

```

1 # 更新数据upsert, autoid未开启
2 # 数据
3 data = [
4     {"my_id": 0, "dense": [0.3580376395471989, -0.6023495712049978,
0.18414012509913835, -0.26286205330961354, 0.9029438446296592], "sparse": {1:
0.5, 100: 0.3, 500: 0.8}, "text": "这是一个例子。"},
5     {"my_id": 1, "dense": [0.19886812562848388, 0.06023560599112088,
0.6976963061752597, 0.2614474506242501, 0.838729485096104], "sparse": {10:
0.1, 200: 0.7, 1000: 0.9}, "text": "这又是一个例子。"}
6 ]
7 # 更新数据
8 res = client.upsert(
9     collection_name="collection1",
10    data=data
11 )

```

6.6 向量匹配search

输入向量，返回匹配的k个数据，参数：

- collection_name: 检索的collection
- anns_field: 检索的field
- data: 输入的向量。需要是二维列表
- limit: 返回的结果数量
- search_params.metric_type: 选填。搜索metric，优先级比field本身的metric_type高。如
`search_params = {"metric_type": "COSINE"}`
- output_fields: 选填。返回的字段内容。默认只返回id和metric值。如 `output_fields = ["text", "sparse"]`

代码块

```
1  # 待查询向量
2  query_vector = [0.3580376395471989, -0.6023495712049978, 0.18414012509913835,
                  -0.26286205330961354, 0.9029438446296592]
3  # search
4  res = client.search(
5      collection_name="collection1",
6      anns_field="dense",
7      data=[query_vector],
8      limit=3,
9      search_params={"metric_type": "COSINE"},
10     output_fields=["text"]
11 )
12 print(res)
```

6.7 条件检索get、query

- 使用get来根据主键id检索数据
- 使用query来根据filter条件表达式筛选数据 (<https://milvus.io/docs/zh/boolean.md>)

可选参数如output_fields、limit

代码块

```
1  # get
2  res = client.get(
3      collection_name="collection1",
4      ids=[463612493562455677, 463612493562455678],
5      output_fields=["my_id", "text"]
6  )
7  print(res)
```



```

1 代码块 query
2  res = client.query(
3      collection_name="collection1",
4      filter="text like '%又是%' AND my_id > 463612493562455677", # text包含“又
      是”，且my_id大于463612493562455677
5      limit=1,
6      output_fields=["my_id", "text"]
7  )
8  print(res)

```

6.8 search、query可以搭配filter与schema实现`文本匹配`、`短语匹配`

在schema对字符串对应field定义 `enable_analyzer` 与 `enable_match` 后，便可以在search、query用filter实现具体文本匹配与短语匹配

可以自选是否定义 `analyzer_params`：配置分析器。type默认为standard，还可选english、chinese。用于辅助分词和去除停用词，提高匹配效果。

- `filter = "TEXT_MATCH(field_name, text)"`
- `filter = "PHRASE_MATCH(field_name, phrase, slop)"` # slop为phrase中允许出现的额外编辑数量，最小为0，最大为2。控制phrase灵活度。

启动匹配会同时自动为field构建倒排索引

代码块

```

1  # 在一个新collection进行该操作
2  schema = MilvusClient.create_schema()
3  schema.add_field(field_name="id", datatype=DataType.INT64, is_primary=True,
4  auto_id=False)
5
6  # 必须先先在schema中定义字段，才能进行文本匹配、短语匹配
7  # 定义字段时，定义enable_analyzer=True,enable_match=True与analyzer_params (可选)
8  schema.add_field(
9      field_name="text",
10     datatype=DataType.VARCHAR,
11     max_length=1024,
12     enable_analyzer=True,
13     enable_match=True,
14     analyzer_params={
15         "type": "chinese"
16     }
17 )
18

```

```

19 client.create_collection(
20     collection_name="match_test",
21     schema=schema,
22     consistency_level="Strong"
23 )
24
25 # index可以随时添加 (所有向量field必须有自己的index)
26 index_params = MilvusClient.prepare_index_params()
27 index_params.add_index(
28     field_name="dense",
29     index_name="dense_index",
30     index_type="AUTOINDEX",
31     metric_type="COSINE"
32 )
33 client.create_index(
34     collection_name="match_test",
35     index_params=index_params
36 )
37
38 # 确保启动加载collection
39 client.load_collection(collection_name="match_test")
40
41 # 先写入两个数据
42 data = [
43     {"id": 0, "dense": [0.3580376395471989, -0.6023495712049978,
44     0.18414012509913835, -0.26286205330961354, 0.9029438446296592], "text": "这是是
45     一个例子。"},
46     {"id": 1, "dense": [0.19886812562848388, 0.06023560599112088,
47     0.6976963061752597, 0.2614474506242501, 0.838729485096104], "text": "这又是一个例
48     子。"}
49 ]
50 res = client.upsert(
51     collection_name="match_test",
52     data=data
53 )
54 print(res)

```

代码块

```

1 # 以query做文本匹配示例
2 res = client.query(
3     collection_name="match_test",
4     filter="not TEXT_MATCH(text, '又') and TEXT_MATCH(text, '例子')",
5     output_fields=["text"]
6 )
7 print(res)

```

```

8
9 # 以query做短语匹配示例（不知道为什么这里只能匹配到一项，可能是分词原因）
10 res = client.query(
11     collection_name="match_test",
12     filter="PHRASE_MATCH(text, '这又是', 2)",
13     output_fields=["text"]
14 )
15 print(res)

```

7. 使用Milvus内置粗召回

7.1 BM25关键词匹配

需要预设schema、index，在schema中加入bm25的function。

schema中的bm25 function，能够自动在指定field中生成对应的稀疏向量。传入时便只需要在待转换field中传入文本。

- 待转换文本field必须启动 `enable_analyzer`，才能正常使用bm25 function。可以配合加上 `analyzer_params = {"type": "chinese"}`，指定文本分析器。
- 自动转换的稀疏向量只能用于检索，不能输出（缺陷：无法用output_fields提取出来直接使用）。

代码块

```

1 from pymilvus import MilvusClient, DataType, Function, FunctionType
2 client = MilvusClient(uri="http://localhost:19530", token="root:Milvus")

```

schema需要定义

```

1 schema = client.create_schema()
2 schema.add_field(field_name="id", datatype=DataType.INT64, is_primary=True,
3     auto_id=True)
4 # 待转换文本列
5 schema.add_field(field_name="text", datatype=DataType.VARCHAR,
6     max_length=1024, enable_analyzer=True, analyzer_params={"type": "chinese"})
7 # 保存自动转换的稀疏向量列
8 schema.add_field(field_name="bm25_sparse",
9     datatype=DataType.SPARSE_FLOAT_VECTOR)

```

schema需要定义

```

1 # 定义bm25的向量转换函数
2 bm25_function = Function(
3     name="text_bm25_emb",

```

```

4     input_field_names=["text"], # 待转换列
5     output_field_names=["bm25_sparse"], # 保存自动转换的稀疏向量列
6     function_type=FunctionType.BM25, # 函数类型为bm25
7 )
8 schema.add_function(bm25_function)

```

代码块

```

1 # 配置索引 (详见milvus数据管理文档)
2 index_params = client.prepare_index_params()
3 index_params.add_index(
4     field_name="bm25_sparse",
5     index_type="SPARSE_INVERTED_INDEX",
6     metric_type="BM25",
7     params={
8         "inverted_index_algo": "DAAT_MAXSCORE",
9         "bm25_k1": 1.2,
10        "bm25_b": 0.75
11    }
12 )

```

代码块

```

1 client.create_collection(
2     collection_name="bm25_collection",
3     schema=schema,
4     index_params=index_params,
5     consistency_level="Strong"
6 )
7 client.load_collection(collection_name="bm25_collection")

```

插入文本数据便会自动转换为稀疏向量，并保存到bm25_sparse列中。

代码块

```

1 res= client.insert(
2     collection_name="bm25_collection",
3     data=[
4         {"text": "这是一个文档"},
5         {"text": "这是另一个文档"},
6     ]
7 )
8 print(res)

```

检索匹配目标列时，便会按照bm25算法进行匹配

- 请勿返回bm25_sparse列，自动转换的稀疏向量不支持检索返回

代码块

```
1 res = client.search(  
2     collection_name="bm25_collection",  
3     data=["这是另外的文档"],  
4     anns_field="bm25_sparse",  
5     output_fields=["text"],  
6     limit=2  
7 )  
8 print(res)
```

7.2 BGE-M3语义匹配（基于HuggingFace TEI）

- 首先使用HuggingFace官方方法，将TEI作为独立服务部署（保障灵活性与控制权）。使用docker部署并保存TEI服务端点。参考https://huggingface.co/docs/text-embeddings-inference/en/quick_tour#deploy

更多TEI部署可参考 [📖 windows+docker desktop部署HF TEI（GPU）](#)

- 待转换文本列需要控制字数。bge-m3最大token为8194，建议varchar的max_length同样为8194。
- bge-m3模型输出为1024维向量，建议float_vector的dim为1024。
- 定义嵌入函数时，通常参数包括：
 - provider**：必须。TEI方式为TEI
 - endpoint**：必须。TEI服务port，例如<http://localhost:8080>。当milvus在docker中运行时，想访问宿主机的端口服务，需要使用<http://host.docker.internal:8080>
 - truncate**：可选。截断超出模型嵌入长度的文本。"true"/"false"（default）
 - truncation_direction**：可选。截断方向。"Left"/"Right"（default）
 - max_client_batch_size**：Milvus客户端发送到TEI的最大批量大小。默认为32。
 - prompt_name**：高级可选。用于某些需要特定提示格式的模型。"your_prompt_key"
 - ingestion_prompt**：高级可选。指定在数据插入（摄取）阶段使用的提示。"passage："
 - search_prompt**：高级可选。指定在搜索阶段使用的提示。"query："
- 自动转换的向量同样无法在检索时导出。

代码块

```
1 from pymilvus import MilvusClient, DataType, Function, FunctionType,
```

CollectionSchema, FieldSchema

```
2 client = MilvusClient(uri="http://localhost:19530", token="root:Milvus")
```

代码块

```
1 schema = MilvusClient.create_schema()
2 schema.add_field(field_name="id", datatype=DataType.INT64, is_primary=True,
  auto_id=True)
3 schema.add_field(field_name="text", datatype=DataType.VARCHAR,
  max_length=8194) # 注意文本长度
4 schema.add_field(field_name="dense", datatype=DataType.FLOAT_VECTOR, dim=1024)
  # 注意向量维度
```

代码块

```
1 # 定义embedding函数
2 text_embedding_function = Function(
3     name="bge-m3-func",
4     function_type=FunctionType.TEXTEMBEDDING,
5     input_field_names=["text"],
6     output_field_names=["dense"],
7     params={
8         "provider": "TEI",
9         "endpoint": "http://host.docker.internal:8080",
10        "truncate": "true",
11        "truncation_direction": "Right",
12        # "max_client_batch_size": 64,
13    }
14 )
15 schema.add_function(text_embedding_function)
```

代码块

```
1 index_params=client.prepare_index_params()
2 index_params.add_index(
3     field_name="dense",
4     index_type="AUTOINDEX",
5     metric_type="COSINE"
6 )
```

代码块

```
1 client.create_collection(
2     collection_name="bge_m3_test",
```

```

3     schema=schema,
4     index_params=index_params,
5     consistency_level="Strong",
6 )
7 client.load_collection(collection_name="bge_m3_test")

```

代码块

```

1 res = client.insert(
2     collection_name='bge_m3_test',
3     data=[{'text': 'Milvus simplifies semantic search through embeddings.'},
4           {'text': 'Vector embeddings convert text into searchable numeric data.'},
5           {'text': 'Semantic search helps users find relevant information quickly.'}]
6 )
7 print(res)

```

代码块

```

1 # 检索向量
2 results = client.search(
3     collection_name='bge_m3_test',
4     data=['How does Milvus handle semantic search?'], # 可以直接用原始文本检索，也
5     # 可以使用嵌入向量
6     anns_field='dense',
7     limit=2,
8     output_fields=['text'],
9 )
10 print(results)

```

7.3 RRF混合检索（以BM25关键词+BGE-M3语义混合为例）

Milvus中，混合检索通常通过混合单独的 `AnnSearchRequest` 对象来实现，每个对象对应一个检索策略（如BM25或BGE-M3）。

基于search对象，使用 `hybrid_search` 方法进行混合检索。混合检索方案以 `Function` 对象作为方法的传入参数。

rrf function需要定义的参数 `params = {}` 包括：

- `"reranker": "rrf"`。其实严谨来说，**RRF并非reranker，而是组合策略检索算法**。Milvus设置的 `hybrid_search.reranker` 其实限制了真正的混合检索算法和reranker的组合使用，可能使特殊情况下在此处使用真正的 `reranker` 无法起到混合检索的作用。个人认为 `hybrid_search` 的 `reranker` 应当变更名称

- "k": 60。k为平滑参数，默认值为60，范围(0, 16384)。k值越小，更重视精英排名，第一第五差距巨大。k值越大，更重视民主排名，第一第五差距更小。

rfr function的 `input_field_names=[]` (!!!!混合检索需要用的function都是如此)

混合检索输出字段参数名为 `output_fields` 而非 `output_field_names`

代码块

```
1 from pymilvus import MilvusClient, DataType, Function, FunctionType
2 client = MilvusClient(uri="http://localhost:19530", token="root:Milvus")
```

代码块

```
1 # 这里分别创建BM25的spasevector、bge的densevector、支持bm25的text的schema
2 schema = MilvusClient.create_schema()
3
4 schema.add_field(field_name="id", datatype=DataType.INT64, is_primary=True,
5 auto_id=True)
6 schema.add_field(field_name="text", datatype=DataType.VARCHAR,
7 max_length=8194, enable_analyzer=True)
8
9 schema.add_field(field_name="sparse", datatype=DataType.SPARSE_FLOAT_VECTOR)
10 schema.add_field(field_name="dense", datatype=DataType.FLOAT_VECTOR, dim=1024)
11
12 bm25_function = Function(name="bm25_func", function_type=FunctionType.BM25,
13 input_field_names=["text"], output_field_names=["sparse"])
14 bge_function = Function(name="bge-m3-func",
15 function_type=FunctionType.TEXTEMBEDDING, input_field_names=["text"],
16 output_field_names=["dense"],
17 params={
18     "provider": "TEI",
19     "endpoint": "http://host.docker.internal:8080",
20     "truncate": "true"
21 })
22
23 schema.add_function(bm25_function)
24 schema.add_function(bge_function)
```

代码块

```
1 # 创建索引
2 index_params = client.prepare_index_params()
3 index_params.add_index(field_name="sparse", index_name="sparse_index",
4 index_type="SPARSE_INVERTED_INDEX", metric_type="BM25",
5 params={
6     "inverted_index_algo": "DAAT_MAXSCORE",
7 })
```



```

6     "bm25_k1": 1.2,
7     "bm25_b": 0.75
8 })
9 index_params.add_index(field_name="dense", index_name="dense_index",
    index_type="AUTOINDEX", metric_type="COSINE")

```

代码块

```

1 client.create_collection(
2     collection_name="hybrid_test",
3     schema=schema,
4     index_params=index_params,
5     consistency_level="Strong",
6 )
7 client.load_collection(collection_name="hybrid_test")

```

代码块

```

1 data = [{"text": "Red cotton t-shirt with round neck"}, {"text": "Wireless
    noise-cancelling over-ear headphones"}, {"text": "Stainless steel water
    bottle, 500ml"}]
2 client.insert(collection_name="hybrid_test", data=data)

```

创建多个 `AnnSearchRequest` 实例（其实就是search实例，因为在milvus中默认search均为ann）

* 实例的参数与普通search大致相同（不完全相同），后者参考milvus_数据管理

* param中可以重新定义metric_type，定义nprobe数量等

- 如果还需要结合hybridsearch，不需要在当前实例中设置output_field_names
- 此处常用参数包括：
 - data：必须。待检索数据，向量或文本等
 - anns_field：必须。检索的field
 - param：必须。搜索参数。如param = {"metric_type": "COSINE", "nprobe": 3}。如果已经定义好不需要设置，可以使用{}
 - limit：必须。返回的结果数量

* expr：可选。过滤条件（和filter相同）。如expr = "id in [1, 2, 3]"

代码块

```

1 from pymilvus import AnnSearchRequest
2 query_text = "white headphones, quiet and comfortable"

```

代码块

```
1 dense_search = AnnSearchRequest(  
2     data=[query_text],  
3     anns_field="dense",  
4     param={},  
5     limit=3,  
6 )  
7 sparse_search = AnnSearchRequest(  
8     data=[query_text],  
9     anns_field="sparse",  
10    param={},  
11    limit=3,  
12 )  
13 reqs = [dense_search, sparse_search]
```

定义rrf的function，但需注意，其传入hybrid_search中

代码块

```
1 rrf_func = Function(  
2     name="rrf",  
3     input_field_names=[], # 必须为空  
4     function_type=FunctionType.RERANK,  
5     params={  
6         "reranker": "rrf",  
7         "k": 60  
8     }  
9 )
```

进行hybrid_search，可以在这里设置之前没有设置的**output_fields**（不是output_field_names）

代码块

```
1 hybrid_results = client.hybrid_search(  
2     collection_name="hybrid_test",  
3     reqs=reqs,  
4     ranker=rrf_func,  
5     output_fields=["text"],  
6     limit=2  
7 )  
8 print(hybrid_results)
```

8. 使用Milvus内置精召回

8.1 BGE-RERANKER-BASE深度语义匹配

- 直接利用attention机制的token级语义联系，实现深入语义匹配精召回。
- 通常在粗召回（语义、关键词、混合）初筛后，再用reranker对结果精排。
- 在milvus中，reranker（严格意义上的reranker）可以用于直接向量检索与混合检索。其在混合检索中作为function其实是不太合理的：①在混合检索中使用意味着可能要跑重复数据，不符合reranker的工程使用思路（very expensive）②因其只反映语义匹配，在特殊情况下（比如参考了关键词）无法起到混合多路检索的作用（无法公正对待关键词匹配）
- reranker可作为function传入 `search` 或 `hybrid_search`（不建议后者，因此没有举例。可查看官方文档或模仿milvus_粗召回.RRF混合检索），也可以 自行调用模型（见RRF + BGE-Reranker）。

代码块

```
1  from pymilvus import MilvusClient, DataType, Function, FunctionType,
   AnnSearchRequest
2
3  client = MilvusClient(uri="http://localhost:19530", token="root:Milvus")
4
5  # 这里分别创建BM25的spasevector、bge的densevector、支持bm25的text的schema
6  schema = MilvusClient.create_schema()
7
8  schema.add_field(field_name="id", datatype=DataType.INT64, is_primary=True,
   auto_id=True)
9  schema.add_field(field_name="text", datatype=DataType.VARCHAR,
   max_length=8194, enable_analyzer=True)
10 schema.add_field(field_name="sparse", datatype=DataType.SPARSE_FLOAT_VECTOR)
11 schema.add_field(field_name="dense", datatype=DataType.FLOAT_VECTOR, dim=1024)
12
13 bm25_function = Function(name="bm25_func", function_type=FunctionType.BM25,
   input_field_names=["text"], output_field_names=["sparse"])
14 bge_function = Function(name="bge-m3-func",
   function_type=FunctionType.TEXTEMBEDDING, input_field_names=["text"],
   output_field_names=["dense"],
15 params={
16     "provider": "TEI",
17     "endpoint": "http://host.docker.internal:8080",
18     "truncate": "true"
19 })
20
21 schema.add_function(bm25_function)
22 schema.add_function(bge_function)
```

```

23
24 # 创建索引
25 index_params = client.prepare_index_params()
26 index_params.add_index(field_name="sparse", index_name="sparse_index",
27 index_type="SPARSE_INVERTED_INDEX", metric_type="BM25",
28 params={
29     "inverted_index_algo": "DAAT_MAXSCORE",
30     "bm25_k1": 1.2,
31     "bm25_b": 0.75
32 })
33 index_params.add_index(field_name="dense", index_name="dense_index",
34 index_type="AUTOINDEX", metric_type="COSINE")
35
36 # 创建collection
37 client.create_collection(
38     collection_name="reranker_test",
39     schema=schema,
40     index_params=index_params,
41     consistency_level="Strong",
42 )
43 client.load_collection(collection_name="reranker_test")
44
45 # 写入数据
46 data = [{"text": "Red cotton t-shirt with round neck"}, {"text": "Wireless
47 noise-cancelling over-ear headphones"}, {"text": "Stainless steel water
48 bottle, 500ml"}]
49 client.insert(collection_name="reranker_test", data=data)

```

search中定义BGE-Reranker-Base的function（基于HuggingFace TEI）

- function参数包括：
 - name：函数名称
 - input_field_names：输入字段名称
 - function_type：FunctionType.RERANK
 - params：
 - "queries": ["需要匹配的query"]
 - "reranker": "model"。此处填写固定字段model
 - "provider": "tei"。此处服务为tei
 - "endpoint": "<http://host.docker.internal:8081>"。此处填写TEI服务地址（区分localhost与host.docker.internal）
 - "truncate": True。可选，是否截断文本。

- "truncation_direction": "Right"。可选，截断方向，默认Right。
 - "max_client_batch_size": 32。可选，最大客户端批次大小，默认32。
- 传入search，根据初筛结果进行精排序

代码块

```
1 query_text = "white headphones, quiet and comfortable"
```

代码块

```
1 bge_rerank_func = Function(  
2     name="bge_rerank_func",  
3     input_field_names=["text"],  
4     function_type=FunctionType.RERANK,  
5     params={  
6         "queries": [query_text],  
7         "reranker": "model",  
8         "provider": "tei",  
9         "endpoint": "http://host.docker.internal:8081",  
10        "truncate": True,  
11        # "truncation_direction": "Right",  
12        # "max_client_batch_size": 32,  
13    }  
14 )
```

BGE-M3 + BGE-Reranker-Base (search)

代码块

```
1 results = client.search(  
2     collection_name="reranker_test",  
3     data=[query_text],  
4     anns_field="dense",  
5     limit=2, # 三个里面先筛出两个，仅作示例  
6     output_fields=["text"],  
7     ranker=bge_rerank_func, # 传入reranker函数负责对bgem3选出的两个重新精排  
8     consistency_level="Strong"  
9 )  
10 print(results)
```

BM25 + BGE-Reranker-Base (search)

代码块

```

1  results2 = client.search(
2      collection_name="reranker_test",
3      data=[query_text],
4      anns_field="sparse",
5      limit=2,
6      output_fields=["text"],
7      ranker=bge_rerank_func,
8      consistency_level="Strong"
9  )
10 print(results2)

```

RRF (BM25 + BGE-M3) + BGE-Reranker-Base (自行调用)

hybrid_search初筛后，自行调用模型精排

自行调用模型有三种方式：

- 继续使用Hugging Face TEI的服务端口
- 使用milvus配置好的模型库
- 本地pytorch调用hugging face模型

代码块

```

1  # 混合检索先召回结果，3筛2（仅示例）
2  dense_search = AnnSearchRequest(
3      data=[query_text],
4      anns_field="dense",
5      param={},
6      limit=3,
7  )
8  sparse_search = AnnSearchRequest(
9      data=[query_text],
10     anns_field="sparse",
11     param={},
12     limit=3,
13 )
14 reqs = [dense_search, sparse_search]
15
16 rrf_func = Function(
17     name="rrf",
18     input_field_names=[], # 必须为空
19     function_type=FunctionType.RERANK,
20     params={
21         "reranker": "rrf",
22         "k": 60
23     }

```

```

24 )
25
26 hybrid_results = client.hybrid_search(
27     collection_name="reranker_test",
28     reqs=reqs,
29     ranker=rrf_func,
30     output_fields=["text"],
31     limit=2
32 )

```

代码块

```

1  # 用于整合混合检索返回的数据格式
2  def parse_results(hybrid_results):
3      results = []
4      res_dicts = hybrid_results[0]
5      for res_dict in res_dicts:
6          current_dict = {}
7          for key, value in res_dict.items():
8              if key == 'entity':
9                  entity = res_dict['entity']
10                 if entity:
11                     for ent_key, ent_value in entity.items():
12                         current_dict[ent_key] = ent_value
13             else:
14                 current_dict[key] = value
15             results.append(current_dict)
16     return results

```

代码块

```

1  results_hybrid = parse_results(hybrid_results)
2  print(results_hybrid)
3  documents = [item['text'] for item in results_hybrid]
4  print(documents)

```

使用TEI服务（自行调用）

需要提供接口地址，例如: <http://localhost:8081/rerank>

通常接口服务处，嵌入为 `/embed`，reranker为 `/rerank`

对reranker来说，请求体常用参数包括 `query` (str)、`texts` (List[str])、`truncate` (bool)、`return_text` (bool)

```
1 import requests
2 import json
```

代码块

```
1 # 调用TEI服务的reranker接口
2 reranker_url = "http://localhost:8081/rerank"
3 # 请求体
4 payload = {
5     "query": query_text,
6     "texts": documents,
7     "truncate": True,
8     "return_text": True
9 }
```

代码块

```
1 try:
2     response = requests.post(url=reranker_url, json=payload)
3     response.raise_for_status()
4     results = response.json()
5     print(results)
6 except Exception as e:
7     print(f"Error: {e}")
```

使用Milvus模型库（自行调用）

参考 `milvus_便捷模型调用.ipynb`

milvus提供了丰富的embed（<https://milvus.io/docs/zh/embeddings.md>）与rerank模型（<https://milvus.io/docs/zh/rerankers-overview.md>）

此处我们使用官方接口调用BGE-Reranker-Base:

- 需要使用 `BGERerankFunction(model_name, device)` 初始化实例
- 调用实例时，需要传入query文本、documents列表、top_k（可选）

代码块

```
1 from pymilvus.model.reranker import BGERerankFunction
```

代码块

```
1 bge_rf = BGERerankFunction(
2     model_name="BAAI/bge-reranker-base",
```



```
3     device="cuda:0"
4 )
5
6 results = bge_rf(
7     query=query_text,
8     documents=documents,
9     # top_k=3,
10 )
11 print(results)
```